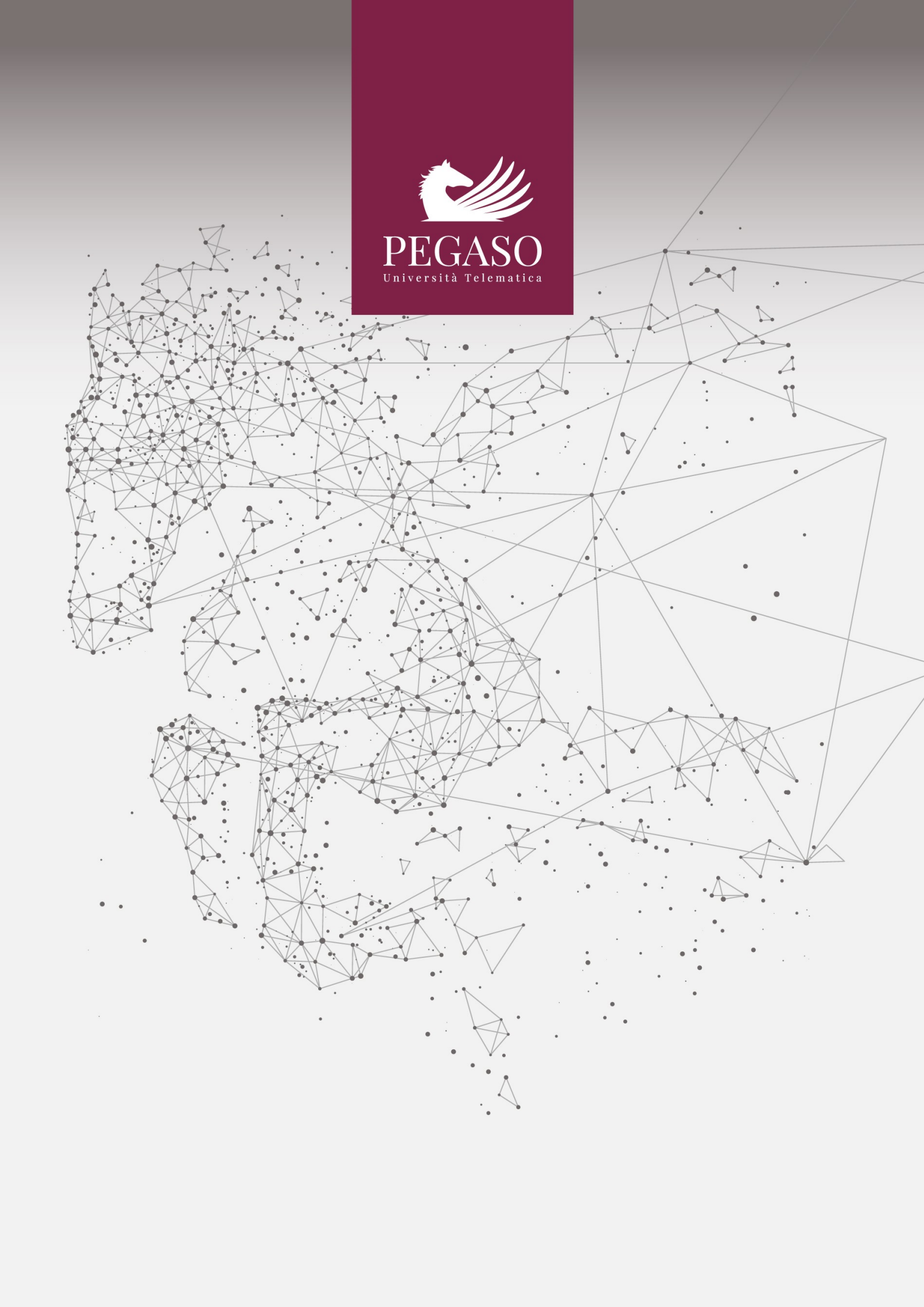

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE 3

2. GENERAZIONE DI NUMERI CASUALI 4

2.1 DISTRIBUZIONE UNIFORME 5
2.2 DISTRIBUZIONE NORMALE 9
2.3 DISTRIBUZIONE BINOMIALE 12

3. CALCOLO DELLE MISURAZIONI STATISTICHE 14

4. COVARIANZA E CORRELAZIONE 19

BIBLIOGRAFIA 24

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

In questa dispensa di accompagnamento alla video-lezione, si presentano le funzioni del modulo numpy.random per la generazione di numeri casuali per le principali distribuzioni di probabilità: uniforme, normale e binomiale. Vengono, poi, presentate le funzioni per il calcolo delle misurazioni statistiche offerte da Python (funzioni built-in), dal modulo statistics e da numpy (per gli array). Infine, vengono illustrati i concetti di covarianza e correlazione e le funzioni che permettono di calcolare la matrice di covarianza e l'indice di correlazione di Pearson. Infine, vengono presentati alcuni esempi di calcolo di covarianza e correlazione per piccole sequenze di dati.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Generazione di numeri casuali

Il supporto di Python per la probabilità e la statistica è suddiviso su più moduli:

1. **numpy e numpy.random;**
2. **statistics;**
3. **scipy.stats** (non analizzato in questa lezione).

Anche **Pandas** (che vedremo tra alcune lezioni) offre alcune funzionalità per la probabilità e la statistica.

Il modulo **numpy.random** offre dei generatori di numeri casuali per tutte le principali **distribuzioni di probabilità.**

Prima di iniziare è fondamentale ricordare che il codice di analisi dei dati dovrebbe essere riproducibile, cioè chiunque dovrebbe essere in grado di lanciare lo stesso programma con gli stessi dati di input e ottenere gli stessi risultati. Per poter ottenere ciò è necessario inizializzare sempre il seme pseudo-casuale con la funzione **seed**(x) scegliendo lo stesso valore di x. In caso contrario i generatori di numeri casuali produrranno ad ogni esecuzione del programma sequenze pseudo-casuali differenti ottenendo, così, risultati differenti come osservabile nel codice seguente in cui nelle prime due chiamate a uniform() viene invocato seed(1), mentre nella terza chiamata a uniform(), la funzione seed() non viene invocata e viene fornito un valore diverso.

```
import numpy.random as rnd
rnd.seed(1)
print(rnd.uniform(low=0.0, high = 1.0, size=None))
rnd.seed(1)
print(rnd.uniform(low=0.0, high = 1.0, size=None))
print(rnd.uniform(low=0.0, high = 1.0, size=None))
```

```
0.417022004702574
0.417022004702574
0.7203244934421581
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Solo a titolo di ripasso si ricorda che una **distribuzione delle probabilità** assegna una probabilità a ogni possibile numero casuale (in una distribuzione discreta) o a un intervallo di numeri casuali (in una distribuzione continua). In altre parole, ci dice se determinati risultati sono più probabili di altri.

Fra le distribuzioni delle proprietà più comuni vi sono:

- la **distribuzione uniforme** (continua e discreta);
- la **distribuzione normale** (continua);
- la **distribuzione binomiale** (discreta).

È possibile specificare una distribuzione delle probabilità fornendo una **funzione di massa della probabilità** (PMF, per le distribuzioni discrete) o una **funzione di densità della probabilità** (PDF, per le distribuzioni continue).

2.1 Distribuzione uniforme

La distribuzione uniforme è una distribuzione nella quale tutti risultati sono ugualmente probabili. Le distribuzioni uniformi vengono spesso utilizzate per rappresentare variabili casuali di tipo numerico quando tutto ciò che si conosce è l'intervallo dei valori possibili. Il modulo random di NumPy mette a disposizione la funzione **uniform**() per la generazione di numeri casuali reali.

La funzione uniform() riceve come parametri 3 argomenti:

- **low** – l'estremo inferiore dell'intervallo;
- **high** – l'estremo superiore dell'intervallo;
- **size** – il numero dei numeri da generare.

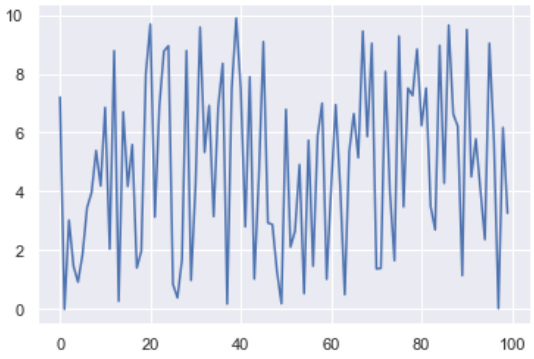
Nel codice seguente, uniform() viene invocata prima per un solo valore (visualizzato a schermo). Successivamente, viene invocata per l'estrazione di 100 valori. Si passa, dunque, a graficare il risultato ottenuto e a visualizzare l'istogramma calcolato sul risultato ottenuto.

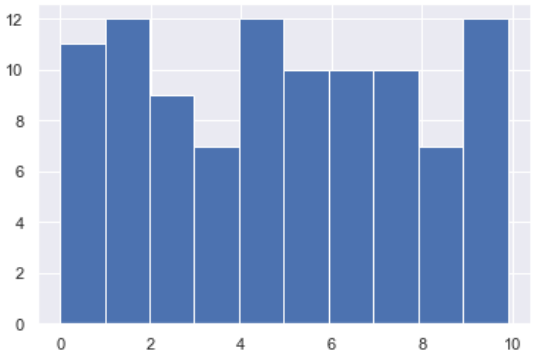
PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
import matplotlib.pyplot as matplot
import numpy.random as rnd
rnd.seed(1)
numeri = rnd.uniform(low=0.0, high=10.0, size=None)
print(numeri)
numeri = rnd.uniform(low=0.0, high=10.0, size=100)
matplot.plot(numeri)
matplot.show()
matplot.hist(numeri)
matplot.show()
```

```
4.17022004702574
```





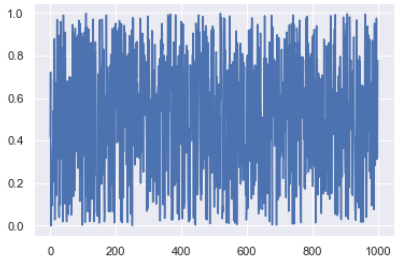
Anche la funzione **rand**(shape) genera numeri casuali secondo la distribuzione uniforme ed equivale a uniform() con i seguenti parametri:

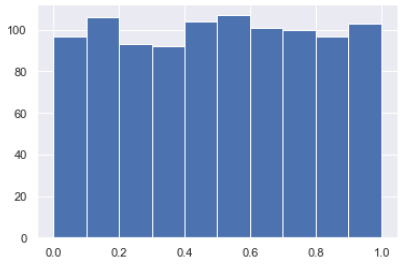
- low = 0.0;
- high = 1.0;
- shape = size.

Nel codice seguente si invoca la funzione rand() per 1.000 elementi.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

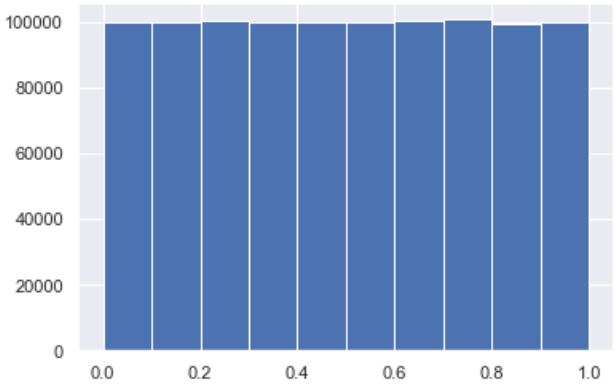
```
import matplotlib.pyplot as matplot
import numpy.random as rnd
rnd.seed(1)
numeri = rnd.rand(1000)
matplot.plot(numeri)
matplot.show()
matplot.hist(numeri)
matplot.show()
```





Al crescere del valore di size, la forma dell'istogramma ottenibile con **hist**() si avvicina alla caratteristica onda quadra della distribuzione uniforme come osservabile nel seguente esempio che genera 1 milione di numeri.

```
import matplotlib.pyplot as matplot
import numpy.random as rnd
rnd.seed(1)
numeri = rnd.rand(1000000)
matplot.hist(numeri)
matplot.show()
```



PEGASO
Università Telematica

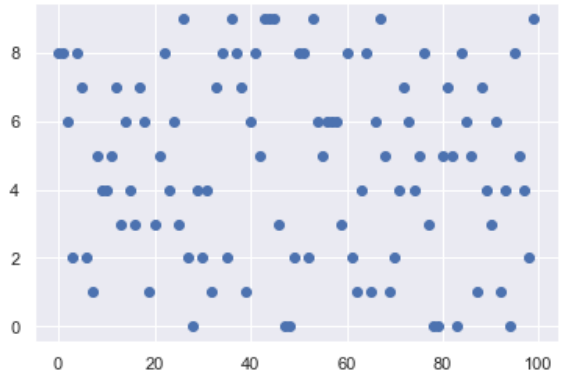
Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

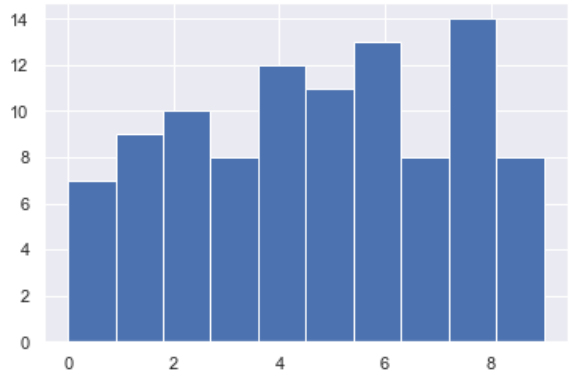
Per i numeri casuali interi a distribuzione uniforme, il modulo random di NumPy mette a disposizione la funzione **randint**(). Anche la funzione randint() riceve come parametri 3 argomenti:

- **low** – l'estremo inferiore dell'intervallo;
- **high** – l'estremo superiore dell'intervallo;
- **size** – il numero dei numeri da generare.

Nel codice seguente, si generano 100 numeri interi nel range [0,9] uniformemente distribuiti.

```
import matplotlib.pyplot as matplot
import numpy.random as rnd
rnd.seed(2)
x = range(100)
numeri = rnd.randint(0,10,size = 100)
matplot.scatter(x,numeri)
matplot.show()
matplot.hist(numeri)
matplot.show()
```

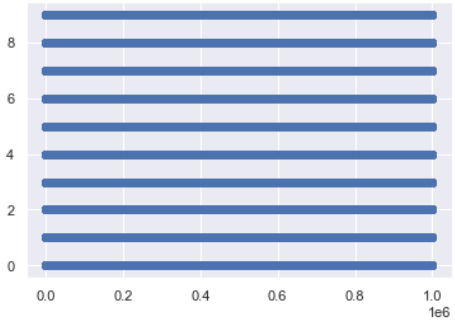


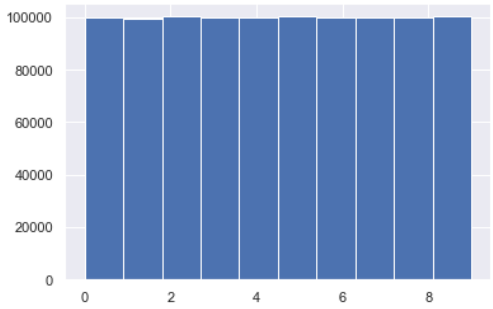


Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

È molto interessante vedere i grafici mostrati precedentemente per un grande valore di size (nel nostro caso 1 milione).

```
import matplotlib.pyplot as matplot
import numpy.random as rnd
rnd.seed(2)
x = range(1000000)
numeri = rnd.randint(0,10,size = 1000000)
matplot.scatter(x,numeri)
matplot.show()
matplot.hist(numeri)
matplot.show()
```





2.2 Distribuzione normale

La **distribuzione normale**, nota anche come distribuzione **gaussiana o a campana**, viene usata per rappresentare i variabili casuali di tipo reale, quando sono note la **media** e la **deviazione standard** della distribuzione, ma non la distribuzione stessa.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il modulo **random** di NumPy mette a disposizione la funzione **normal**() per la generazione di numeri casuali reali. La funzione **normal**() riceve come parametri 3 argomenti:

- **loc** – di default uguale a 0.0 è la media ("centro") della distribuzione;
- **scale** – la deviazione standard ("spread") della distribuzione. Deve essere un numero non negativo (>=0);
- **size** – il numero dei numeri da generare.

La proprietà **bins** (in questo caso 30) di hist() indica il numero degli intervalli per il conteggio.

La proprietà **density=True** indica, invece, che ogni intervallo visualizzerà il conteggio grezzo del bin diviso per il numero totale di conteggi e la larghezza del bin in modo che l'area sotto l'istogramma si integri a 1.

Al fine di visualizzare il grafico della gaussiana, ricordiamo che la funzione di densità della probabilità ha la seguente formula:

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

dove p è la media e s^2 è la varianza.

Nel codice seguente si generano 1.000 numeri distribuiti secondo una gaussiana con media pari a 0 e deviazione standard pari a 0.1. Si procede, quindi, con il calcolo della media e della deviazione standard sui numeri generati per verifica.

Si grafica, poi, l'istogramma dei numeri generati e si sovrappone la curva gaussiana ottenuta mediante funzione di densità della probabilità con media pari a 0 e deviazione standard pari a 0.1.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

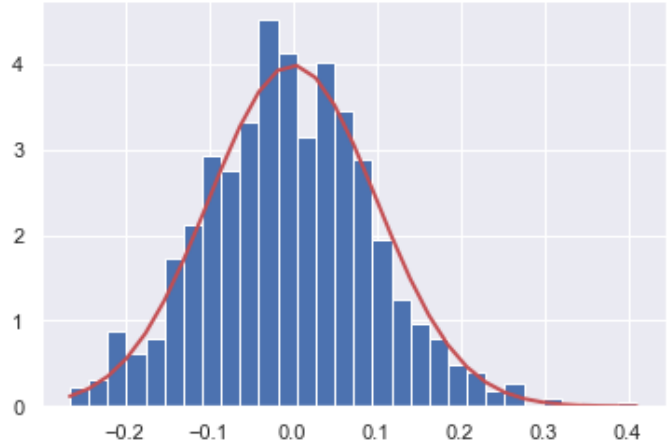
```
import matplotlib.pyplot as matplot
import numpy.random as rnd
rnd.seed(2)
mu, sigma = 0, 0.1 # media e deviazione standard
s = np.random.normal(mu, sigma, 1000)

print("Media della sequenza di numeri",np.mean(s))
print("Varianza della sequenza di numeri",np.std(s, ddof=1))

count, bins, ignored = matplot.hist(s, 30, density=True)
matplot.plot(bins, 1/(sigma * np.sqrt(2 * np.pi)) *
               np.exp( - (bins - mu)**2 / (2 * sigma**2) ),
         linewidth=2, color='r')
matplot.show()
```

Come è possibile osservare dal risultato, la media è quasi zero e la deviazione standard è pari a 0.1.

```
Media della sequenza di numeri -0.004784993519517787
Varianza della sequenza di numeri 0.1004874506883593
```



Anche la funzione **randn**(shape) genera numeri casuali secondo la distribuzione gaussiana ed equivale a normal() con i seguenti parametri:

- loc = 0.0;
- scale = 1.0;
- shape = size.

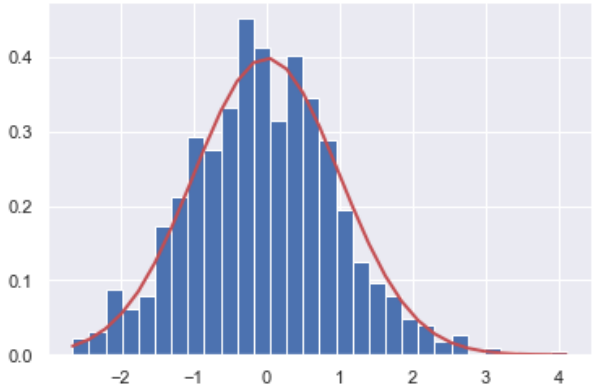
Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
import matplotlib.pyplot as matplot
import numpy.random as rnd
rnd.seed(2)
mu, sigma = 0.0, 1.0 # media e deviazione standard
s = np.random.randn(1000)

print("Media della sequenza di numeri",np.mean(s))
print("Varianza della sequenza di numeri",np.std(s, ddof=1))

count, bins, ignored = matplot.hist(s, 30, density=True)
matplot.plot(bins, 1/(sigma * np.sqrt(2 * np.pi)) *
               np.exp( - (bins - mu)**2 / (2 * sigma**2) ),
         linewidth=2, color='r')
matplot.show()
```

```
Media della sequenza di numeri -0.04784993519517787
Varianza della sequenza di numeri 1.004874506883593
```



2.3 Distribuzione binomiale

La **distribuzione binomiale** è la distribuzione delle probabilità del numero di successi in una sequenza di n>0 prove binarie indipendenti (es. "lanci di monete") ognuno dei quali ha un successo ("testa") con una probabilità $0 \leq p \leq 1$. Il numero atteso di risultati di successo e n*p. Una variabile casuale con distribuzione normale è una variabile a distribuzione binomiale con un numero infinito di prove e una probabilità di successo pari a 0.5.

In altre parole, una variabile casuale con distribuzione normale è data dall'effetto cumulativo di un numero infinito di cause binari equiprobabili. Il modulo random di NumPy mette a disposizione la funzione **binomial**() per la generazione di numeri casuali interi.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La funzione binomial() riceve come parametri 3 argomenti:

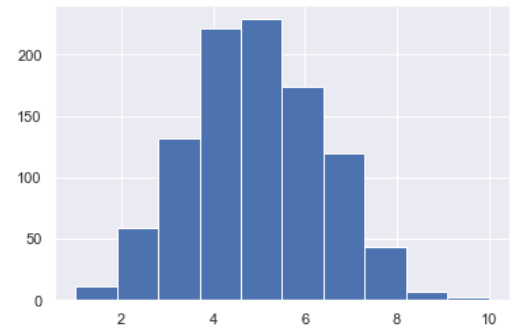
- **n** – il numero di prove;
- **p** – la probabilità della distribuzione 0 ≤ p ≤ 1;
- **size** – il numero di ripetizioni da effettuare.

Nel codice seguente si simula il lancio di una moneta non truccata (p=0.5) per 10 volte, testato su 1.000 ripetizioni. La media risulta essere quasi pari a 0,5 (0,4879). Viene, poi, visualizzato l'istogramma dei risultati generati.

```
%matplotlib inline
import numpy as np
import matplotlib.pyplot as matplot

n, p = 10, .5
#risultato del lancio di una moneta per 10 volte, testato su 1000 ripetizioni
s = np.random.binomial(n, p, 1000)
print(sum(s)/10000)
matplot.hist(s)
matplot.show()
```

```
0.4879
```



PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Calcolo delle misurazioni statistiche

Per procedere con il calcolo delle misurazioni statistiche inizieremo con l'analisi del modulo **statistics** e le **funzioni built-in** di Python. Le prime interessanti misurazioni che si possono svolgere sono riportate nella tabella seguente.

Misurazione	Significato	Come ottenerla
Minimo	Il valore più basso della sequenza	Funzione built-in min()
Massimo	Il valore più alto della sequenza	Funzione built-in max()
Range	Distanza tra il massimo e il minimo	Differenza tra max e min
Media	La somma dei valori divisi per il loro numero	Funzione mean() del modulo statistics
Mediana	Il valore centrale della sequenza una volta che sono stati disposti in ordine non decrescente. Se i valori sono in numero pari, la mediana sarà uguale alla media dei due valori centrali	Funzione median() del modulo statistics
Moda	Il valore che si presenta con maggior frequenza nella sequenza	Funzione mode() del modulo statistics
Varianza	La media dei quadrati delle differenze dei singoli valori e la loro media. Misura la variabilità dei valori presenti nella sequenza	Funzione mean() del modulo statistics
Deviazione standard	La radice quadrata della varianza	Funzione mean() del modulo statistics

Per presentare le funzioni, svolgiamo un semplice esercizio. Ipotizziamo di avere la lista dei voti di uno studente delle scuole superiori di II grado pari a:

voti = [4,8,8,8,9,6,9,7,9,8,3]

Nel complesso lo studente prende quasi sempre 8 e 9, ma ci sono delle "scivolate" lungo il percorso di studi. Andiamo, innanzitutto a graficare i voti e, successivamente a calcolare **media, mediana e varianza.**

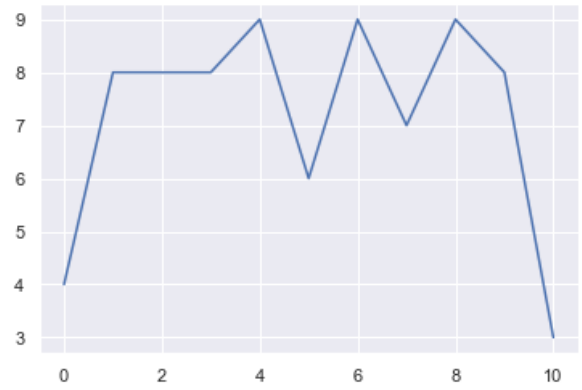
PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
%matplotlib inline
import statistics as s
import matplotlib.pyplot as matplot

voti = [4,8,8,8,9,6,9,7,9,8,3]
matplot.plot(voti)
matplot.show()

print("La media dei voti è:",s.mean(voti))
print("La mediana dei voti è:",s.median(voti))
print("La varianza dei voti è:",s.variance(voti))
```




```
La media dei voti è: 7.181818181818182
La mediana dei voti è: 8
La varianza dei voti è: 4.163636363636364
```

La **media** dei voti è leggermente superiore a 7 (7,18). Hanno pesato i voti molto bassi su un allievo/a che potrebbe avere un 8 di media. La **mediana** pari a 8 rende maggiore "giustizia" alle votazioni (valore centrale della serie ordinata in modo non decrescente).

Già da questi primi risultati è possibile esprimere due considerazioni su queste grandezze:

- la media è fortemente influenzata dai singoli valori;
- la mediana non viene affatto influenzata dagli estremi, ma si basa sul valore centrale presente nella sequenza.

Non a caso, media e mediana vengono spesso esaminati insieme per capire quanto la media possa essere rappresentativa dei dati. La **varianza** rispetto ai valori è piuttosto rilevante.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Considerando che una sequenza di valori tutti uguali ha varianza zero, è possibile capire come sia indicativa della variabilità dei valori considerati. Proviamo, ora, a confrontare questi dati con la stessa lista di voti, ma un po' ristretta, eliminando:

- i due voti più bassi (3 e 4);
- i due voti più alti (due 9).

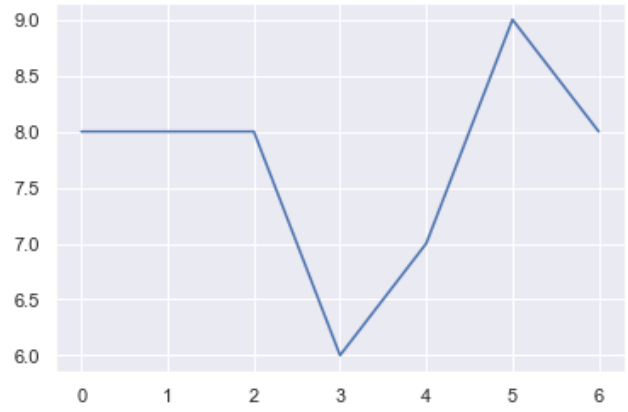
```
%matplotlib inline
import statistics as s
import matplotlib.pyplot as matplot

voti = [8,8,8,6,7,9,8]

print("La media dei voti è:",s.mean(voti))
print("La mediana dei voti è:",s.median(voti))
print("La varianza dei voti è:",s.variance(voti))

matplot.plot(voti)
matplot.show()
```

```
La media dei voti è: 7.714285714285714
La mediana dei voti è: 8
La varianza dei voti è: 0.9047619047619048
```



Osserviamo che scartando i valori estremi (spesso chiamati **outlier** in statistica), otteniamo una media e una mediana più "vicine". La mediana, per sua natura, senza gli elementi estremi rimaste identiche perché sempre un 8 e al centro della sequenza ordinata. La media è stata fortemente influenzata dalla mancanza degli estremi ed è aumentata fino a raggiungere quasi la mediana.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Effettivamente ora sembra aver assunto un valore più consono rispetto l'allievo che stiamo valutando. La varianza, poi, si è ridotta drasticamente. Da un valore di oltre quattro è scesa sotto il valore di uno e questo è segno che la variabilità dei valori è diminuita di molto.

Anche NumPy offre le funzioni aggregazione per gli **array.** Le funzioni di base offerte da NumPy sono **mean**(), **sum**(), **std**() (deviazione standard), **min**() e **max**(). Nel codice seguente si mostra come invocare le funzioni con la libreria numpy.

```
import numpy as np

voti = np.array([4,8,8,8,9,6,9,7,9,8,3])
print("Voti: ",voti)

print("La media dei voti è:",np.mean(voti))
print("La somma dei voti è:",np.sum(voti))
print("La deviazione standard dei voti è:",np.std(voti))
print("Il voto minimo è:",np.min(voti))
print("Il voto massimo è:",np.max(voti))
```

```
Voti:  [4 8 8 8 9 6 9 7 9 8 3]
La media dei voti è: 7.181818181818182
La somma dei voti è: 79
La deviazione standard dei voti è: 1.945539505366609
Il voto minimo è: 3
Il voto massimo è: 9
```

NumPy offre anche due funzioni aggregazione molto interessanti per gli **array**: le funzioni **cumsum**() e **cumprod**(). Tali funzioni applicate ad un array permettono di ottenere un array avente la stessa dimensione dell'array di partenza, ma con gli elementi uguali, rispettivamente, alla somma e al prodotto cumulativo degli elementi di partenza. Come è possibile osservare dal risultato, il primo elemento della somma cumulativa e del prodotto cumulativo è uguale al primo elemento dell'array di partenza.

Il secondo elemento:

- della somma cumulativa è uguale alla somma dei primi due elementi dell'array di partenza (4+8=12);
- del prodotto cumulativo è uguale al prodotto dei primi due elementi dell'array di partenza (4*8=32).

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il terzo elemento:

- della somma cumulativa è uguale alla somma dei primi tre elementi dell'array di partenza (4+8+8=20);
- del prodotto cumulativo è uguale al prodotto dei primi tre elementi dell'array di partenza (4*8*8=256).

E così via per tutti gli altri elementi.

```
import numpy as np

voti = np.array([4,8,8,8,9,6,9])
print("Voti: ",voti)

print("\nArray della somma cumulativa")
print(np.cumsum(voti))
print("\nArray del prodotto cumulativo")
print(np.cumprod(voti))
```

```
Voti:  [4 8 8 8 9 6 9]

Array della somma cumulativa
[ 4 12 20 28 37 43 52]

Array del prodotto cumulativo
[     4     32    256   2048  18432 110592 995328]
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Covarianza e correlazione

La **covarianza** è un valore numerico che fornisce una misura di quanto due variabili varino assieme. Ne valuta, quindi, la loro dipendenza. In statistica, infatti, la covarianza di due variabili statistiche X e Y, indicata come $\sigma_{X,Y}$= Cov(X,Y) è un indice di variabilità congiunta.

Su una popolazione di N osservazioni congiunte (x_i,y_i) di rispettive medie $\bar{x}$ e $\bar{y}$, la covarianza osservata è:

$$\sigma_{X,Y} = \frac{1}{N}\sum_{i=1}^{N}(x_i - \bar{x})(y_i - \bar{y}) = \frac{1}{N}\sum_{i=1}^{N} x_i y_i - \left(\frac{1}{N}\sum_{i=1}^{N} x_i\right)\left(\frac{1}{N}\sum_{i=1}^{N} y_i\right)$$

La **correlazione** è una relazione tra due variabili tale che a ciascun valore della prima corrisponda un valore della seconda, seguendo una certa regolarità. La correlazione è funzione della covarianza e rappresenta valori standardizzati. In base alle caratteristiche presentate, la correlazione può definirsi:

- **diretta** (o positiva) - la variazione di un elemento interessa, in via diretta, anche l'altro (per es. stature alte dei padri corrispondono stature alte dei figli);
- **indiretta** (anche inversa o negativa) - alla variazione di un elemento corrisponde, in senso contrario, quella dell'altro (per es. ad una maggior produzione di grano corrisponde un prezzo minore).

Il grado di correlazione tra due variabili viene espresso tramite l'**indice di correlazione.** Il valore che esso assume è compreso tra −1 (correlazione inversa) e 1 (correlazione diretta e assoluta), con un indice pari a 0 che comporta l'assenza di correlazione. Il valore nullo dell'indice non implica, tuttavia, che le variabili siano indipendenti.

PEGASO
Università Telematica

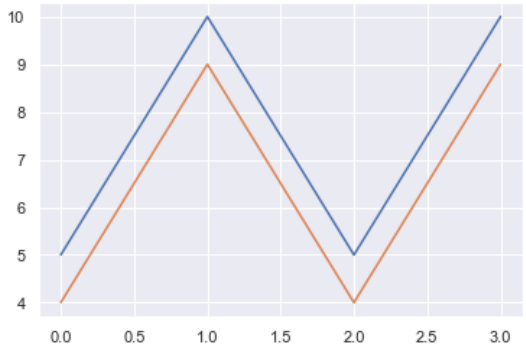
Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

I coefficienti di correlazione sono derivati dagli indici, tenendo presenti le grandezze degli scostamenti dalla media. In particolare, **l'indice di correlazione di Pearson** è calcolato come **rapporto tra la covarianza delle due variabili e il prodotto delle loro deviazioni standard**:

$$-1 \leq \rho_{xy} = \frac{\sigma_{xy}}{\sigma_x \sigma_y} = \frac{\sum_{i=1}^{n}(x_i - \mu_x)(y_i - \mu_y)}{\sqrt{\sum_{i=1}^{n}(x_i - \mu_x)^2}\sqrt{\sum_{i=1}^{n}(y_i - \mu_y)^2}} \leq +1$$

NumPy offre le funzioni per il **calcolo della correlazione e della covarianza.** Per iniziare, definiamo due array fortemente correlati con coefficiente di correlazione uguale ad 1:

```
x = np.array([5,10,5,10])
y = np.array([4,9,4,9])
```



La **matrice di covarianza** restituita dalla funzione **cov**(x,y) di NumPy:

$$((C_{11}\ C_{12})$$

$$(C_{21}\ C_{22}))$$

restituisce:

- sulla diagonale principale (C_{ii}) i valori della covarianza della variabile x_i con sé stessa;
- come valore C_{ij} il valore della covarianza tra x_i e x_j (matrice simmetrica).

  Nel nostro esempio x_i = x e x_j = y.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La **matrice di correlazione** restituita dalla funzione **corrcoef**(x,y) di NumPy:

$$((R_{11}\ R_{12})$$

$$(R_{21}\ R_{22}))$$

restituisce:

- sulla diagonale principale (R_{ii}) i valori della correlazione della variabile x_i con sé stessa;
- come valore R_{ij} il valore della correlazione tra x_i e x_j (matrice simmetrica).

```
%matplotlib inline
import matplotlib.pyplot as matplot
import numpy as np
x = np.array([5,10,5,10])
y = np.array([4,9,4,9])

print("Covarianza (x,y):\n",np.cov(x,y))
print("\nCoefficiente di correlazione (x,y):\n",np.corrcoef(x,y))

matplot.plot(x)
matplot.plot(y)
matplot.show()
```

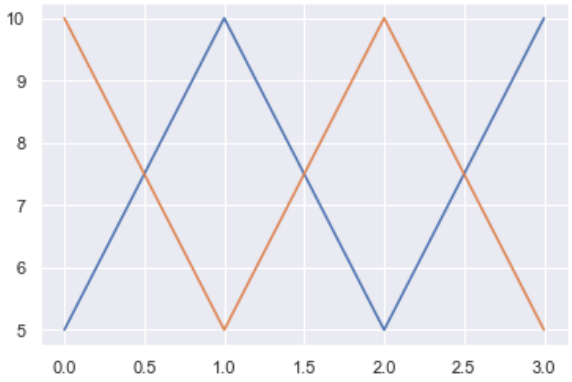
```
Covarianza (x,y):
 [[8.33333333 8.33333333]
 [8.33333333 8.33333333]]

Coefficiente di correlazione (x,y):
 [[1. 1.]
 [1. 1.]]
```

Procediamo con un esempio di variabili inversamente correlate: quando una aumenta, l'altra diminuisce.

```
x = np.array([5,10,5,10])
y = np.array([10,5,10,5])
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).




```
%matplotlib inline
import matplotlib.pyplot as matplot
import numpy as np
x = np.array([5,10,5,10])
y = np.array([10,5,10,5])

print("Covarianza (x,y):\n",np.cov(x,y))
print("\nCoefficiente di correlazione (x,y):\n",np.corrcoef(x,y))

matplot.plot(x)
matplot.plot(y)
matplot.show()
```

Il risultato del coefficiente di correlazione pari a -1 in (0,1) e (1,0) mostra la correlazione inversa tra le due variabili.

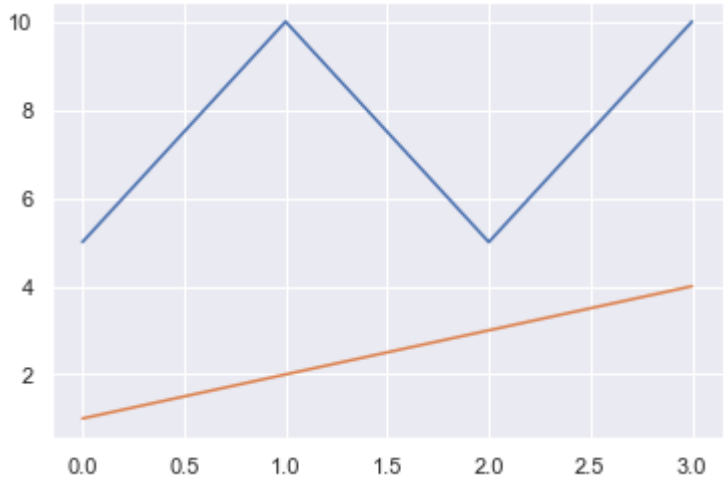
```
Covarianza (x,y):
 [[ 8.33333333 -8.33333333]
 [-8.33333333  8.33333333]]

Coefficiente di correlazione (x,y):
 [[ 1. -1.]
 [-1.  1.]]
```

Procediamo con un esempio di variabili debolmente correlate:

```
x = np.array([5,10,5,10])
y = np.array([1,2,3,4])
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).



Il risultato del coefficiente di correlazione pari a 0,447 in (0,1) e (1,0) mostra la debole correlazione tra le due variabili.

```
%matplotlib inline
import matplotlib.pyplot as matplot
import numpy as np
x = np.array([5,10,5,10])
y = np.array([1,2,3,4])

print("Covarianza (x,y):\n",np.cov(x,y))
print("\nCoefficiente di correlazione (x,y):\n",np.corrcoef(x,y))

matplot.plot(x)
matplot.plot(y)
matplot.show()
```

```
Covarianza (x,y):
 [[8.33333333 1.66666667]
 [1.66666667 1.66666667]]

Coefficiente di correlazione (x,y):
 [[1.        0.4472136]
 [0.4472136 1.       ]]
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Zinoviev D. (2017). Data science con Python. Apogeo.
- Maggi G. (2020). Data science con Python. Edizioni LSWR.
- Marin I., Shukla A., Sarang VK. (2019). L'analisi dei Big Data con Python. Tecniche nuove.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).